

Kamvala

A Location-Based Local Job Matching Application

FWD Course Project - Technical Documentation

Domain: Local job matching for shops in Hyderabad, India

Stack: React 19, TypeScript, Vite, Mapbox, JSON Server

Date: June 2026

1. Introduction

Kamvala is a location-based job discovery application that connects job seekers with nearby shops hiring local help. Unlike large professional networks, Kamvala focuses on everyday work - cashiers, cooks, delivery riders, cleaners - found within walking or short commute distance.

The app is built as a React single-page application displayed inside an iPhone-style phone mockup. It uses a mock REST API (JSON Server) for listings and Mapbox for interactive maps.

2. Problem Statement

Many small shops in Indian cities hire through word-of-mouth or handwritten notices. Job seekers often do not know which shops nearby are hiring, what languages are needed, or what pay is offered.

- Job seekers browse openings on a map within a chosen radius (1-10 km).
- Shop owners enlist their shop and post a job listing with location, pay, and languages.
- Everything stays local - no resume upload, no corporate hiring pipeline.

3. Main Features

Intro Screen (/)

- Landing page with Kamvala brand and two cards: Find a Job and Enlist Shop.

Explore (/explore)

- Mapbox map centred on user GPS or Hyderabad demo area.
- Radius selector: 1, 2, 5, or 10 km.
- Job pins on map; filter by job type, language, and minimum pay.
- Bottom sheet list of nearby jobs.

Job Detail (/explore/:id)

- Full listing with store photo, pay, languages, call shop, get directions.

Enlist (/enlist)

- iOS-style form: shop name, job title, work types, languages, pay, address.
- Optional store image and map pin picker for exact location.
- On submit: listing saved, map centre updated, user navigates to Explore.

4. Technology Stack

- React 19 + TypeScript + Vite 8
- Tailwind CSS v4 with custom iOS-like UI components
- React Router v7 with lazy-loaded pages
- Mapbox GL JS + react-map-gl v8
- JSON Server mock API on port 3001
- Vitest + React Testing Library
- Haversine formula for client-side radius filtering

5. Application Architecture

The app wraps all pages in a PhoneFrame shell. Inside, three React contexts manage global state: LocationContext (map centre from GPS), FilterContext (radius and filters), and ListingsContext (refresh and local listings after enlist).

Services layer: apiClient (fetch wrapper), listingsService (CRUD + merge + filters), and geo (Haversine distance).

Hooks: useGeolocation and useNearbyListings.

6. Routing

- / - Intro (tab bar hidden)
- /explore - Map + job list (Explore | Enlist tabs)
- /explore/:id - Job detail (back button, no tabs)
- /enlist - Shop enlist form

7. API and Data Flow

REST endpoints proxied via Vite at /api:

- GET /listings - all job listings
- GET /listings/:id - single listing
- POST /listings - create listing (shop enlist)
- GET /jobTypes and GET /languages - dropdown options

Enlist to Explore flow

- User submits enlist form with map pin location.
- POST saves listing to db.json (or local fallback if API is down).
- Listing added to ListingsContext immediately for instant display.
- Map centre moves to shop pin; filters cleared; app opens Explore.
- Haversine filter shows only jobs within selected radius.

8. Geolocation and Radius Search

Default centre is Hyderabad (17.385 N, 78.4867 E). If the user allows location, the map uses GPS coordinates. Radius options are 1, 2, 5, and 10 km. Distance is computed with the Haversine formula on the client. A blue circle on the map shows the search radius.

9. Mapbox Integration

- Token in .env as VITE_MAPBOX_TOKEN
- CSP build + web worker for Vite compatibility
- JobMap for explore view; MapPinPicker for enlist form
- Fallback message if no token - rest of app still works

10. UI / UX Design

The interface follows Apple iOS patterns: large titles, grouped lists, segmented controls, frosted tab bar, Dynamic Island phone frame (~430px wide), and hidden scrollbars inside the phone mockup.

11. Project Structure

- src/pages - Intro, Explore, JobDetail, Enlist
- src/components - layout, map, jobs, shop, ui
- src/context - Location, Filter, Listings
- src/services - apiClient, listingsService, geo
- db.json - 10 seeded Hyderabad listings

12. How to Run

- npm install
- cp .env.example .env and add Mapbox token
- npm run dev - Vite on :5173 + JSON Server on :3001
- npm test - run unit tests
- npm run build - production build

13. Course Learning Outcomes (FWD)

- Frontend development - React components, hooks, context, mobile UI
- Routing and navigation - lazy routes and tab shell
- API integration - REST client, POST/GET, proxy, fallback
- Geospatial logic - GPS, Haversine radius, map markers
- State management - multiple React contexts
- Testing - Vitest for geo, listings, forms, and pages
- External services - Mapbox integration with env config

14. Summary

Kamvala is a complete mobile-style job marketplace prototype for Hyderabad. Job seekers explore nearby openings on a map; shop owners enlist with a simple form. The architecture separates UI, state, services, and data, making it

straightforward to extend with a real backend, authentication, or notifications in future work.